

Cheatsheet: Adapt Existing Java Projects

Table of Contents

Configure settings.gradle file.....	1
Adjust gradle.properties	2
Adjust Main build.gradle File.....	3
If you have a single module project....	3
Optional: Documentation Module	4

Configure settings.gradle file

settings.gradle

```
buildscript {
    repositories {
        mavenLocal()
        mavenCentral().content {
            excludeModule("javax.media", "jai_core")
        }
        gradlePluginPortal()
        maven {
            url = uri("https://central.sonatype.com/repository/maven-snapshots")
        }
        maven {
            url = uri("https://mifos.jfrog.io/artifactory/mifosx-gradle-local")
        }
    }

    dependencies {
        classpath "org.mifos:mifos-conventions-gradle-project-java:0.1.0-SNAPSHOT" ①
    }
}

apply plugin: "org.mifos.convention.project.base.layout"

// NOTE: this is where your build modules get included, e.g.:
//
// include(":mifos-myproject")
// project(":mifos-myproject").projectDir = file("${rootDir}/modules/myproject")
```

① This the important part here, dependency on the Mifos build conventions for generic Java projects. If you have a single module Java project then please see the instructions in section xxx.



By default all Java projects are multi-module. This is the most flexible solution.

Adjust gradle.properties

Edit Project Metadata

gradle.properties Project Metadata

```
# project metadata ①
mifos.projects.license.name=MPL-2.0
mifos.projects.license.link=https://spdx.org/licenses/MPL-2.0.html
mifos.projects.homepage=https://mifos.org
mifos.projects.bug.tracker=https://mifosforge.jira.com
mifos.projects.contact=https://mifos.org/about-us/contact-us
mifos.projects.inception.year=2026
mifos.projects.vendor=Mifos Initiative
mifos.projects.copyright=Copyright (c) 2026 Mifos Initiative
mifos.projects.scm.url=https://github.com/openMF/myproject
mifos.projects.scm.connection.main=git@github.com:openMF/myproject.git
mifos.projects.scm.connection.developer=git@github.com:openMF/myproject.git
mifos.projects.contribute=https://github.com/openMF/myproject/CONTRIBUTE.md
mifos.projects.documentation=https://docs.mifos.org/myproject
mifos.projects.donation=https://mifos.org/donations
mifos.projects.faq=https://mifos.org/faq
mifos.projects.help=https://mifos.org
mifos.projects.translate=https://github.com/openMF/myproject/TRANSLATE.md

# contributing developer information ②
mifos.developers.0.username=vidakovic
mifos.developers.0.firstname=Aleksandar
mifos.developers.0.lastname=Vidakovic
mifos.developers.0.email=aleks@mifos.org
mifos.developers.0.url=https://github.com/vidakovic
mifos.developers.0.timezone=UTC+2
mifos.developers.0.organisation=Mifos Initiative
mifos.developers.0.organisationUrl=https://mifos.org
mifos.developers.0.gpg.public.key=
```

① Please adjust to your specific project

② Please put your own information

Edit Gradle Cache Configuration

The new Mifos build conventions allow you to turn on very aggressive caching configurations. All plugins used in the build are able to take advantage of these performance improving settings, except for the Asciidoc plugin. This is not really a constraint as this plugin is not enabled by default when you do a `./gradlew build`. Building the documentation is anyway more important for the

release and not for the day to day builds. See more in the section about the documentation module.

gradle.properties Caching

```
org.gradle.daemon=true
org.gradle.caching=true
org.gradle.configuration-cache=true
org.gradle.configuration-cache.problems=warn
org.gradle.parallel=true
org.gradle.logging.level=quiet
org.gradle.configureondemand=true
```

Adjust Main build.gradle File

build.gradle

```
plugins {
    id "org.mifos.convention.project.base.root" ①
}

description = "Mifos Initiative - My Project" ②
```

① This plugin is important, otherwise the mechanics will not work; this is mostly responsible that all project metadata and developer information is loaded properly and available to all plugins that need it (e.g. Maven publishing).

② Put your own project description here



Nothing else is required in the build.gradle in the root folder. All necessary plugins etc. are configured in the conventions plugin.

If you have a single module project...

If you have a project that has the `src` folder in the project root directory then it's probably a single module project. In this case please execute the following steps:

1. Create a folder `modules` in the project root
2. If your project is called "myproject" then create a folder `modules/myproject`
3. Move the `src` folder from the project root to `modules/myproject/src`
4. Create a `build.gradle` file in `modules/myproject/build.gradle`

Project module build.gradle

```
plugins {
    id "org.mifos.convention.project.java" ①
}
```

```
description = "Mifos Initiative - My Project" ②
```

- ① This plugin activates this project module as a Java project.
- ② Put your own project description here



Keep the folder names under `modules` as simple as possible. Single word names are preferred. If you must then use hyphens (e.g. `my-project`), but try to avoid and keep it as simple as possible. This has no effect on the logic names of your Gradle build modules you define in the `settings.gradle` file; you have all the freedom you need to name them as you see fit.

If you reload the project in your IDE then the following task is executed automatically. Execute the following command once to be sure that everything works and you have the shared configurations for all Mifos projects locally available (you should see a folder `.mifos` in the root of the project):

Unzip Shared Mifos Build Conventions Configuration

```
./gradlew mifosConfigUnzip
```

Finally build the project:

```
./gradlew clean build publish
```

Optional: Documentation Module

1. If your project is called "myproject" then create a folder `modules/doc`
2. Create a `build.gradle` file in `modules/doc/build.gradle`

Project Documentation Module build.gradle

```
plugins {  
    id "org.mifos.convention.project.doc" ①  
}  
  
description = "Mifos Initiative - My Project" ②
```

- ① This plugin activates this project module as an AsciiDoc project.
- ② Put your own project description here



See [here](#) for an example how our AsciiDoc based project modules are structured.



The current version of the Gradle AsciiDoctor plugin is incompatible with Gradle's configuration caching. The command line parameter `--no-configuration-cache` disables that cache.

```
./gradlew asciidoctorZip --no-configuration-cache
```